

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	POLIDAS BHARATH
Reg. No	192425368
GitHub Link	

EXPERIMENTS

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward

Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini- Batch Gradient Descent, Curse of Dimensionality. (BTL- 4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I POLIDAS BHARATH _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student:

QUESTIONS:10

Total Marks:50

CO1- AT3 Experiments
Questions

2. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve. **Answer :**

3. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

4. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix. **Answer :**

5. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

6. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

7. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

1. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

1. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

2. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

3. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

1. Learning Algorithms – Linear Regression Using Gradient Descent

AIM

To implement Linear Regression using Gradient Descent, train the model on a dataset, analyze the decrease in loss over iterations, and plot the learning curve.

ALGORITHM

1. Create input data X and target values y.
2. Initialize weight and bias with zero.
3. Predict output using $y_{\text{pred}} = wX + b$.
4. Calculate Mean Squared Error (MSE).
5. Calculate gradients for weight and bias.
6. Update weight and bias using the learning rate.
7. Repeat the process for a fixed number of iterations.
8. Store loss values during training.
9. Plot the loss iterations against.

PROGRAM

```
import numpy as np

import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 6, 8, 10], dtype=float)

w = 0.0

b = 0.0

learning_rate = 0.01

iterations = 1000
```

```
loss_history = []

n = len(X)

for i in range(iterations):

    y_pred = w * X + b

    loss = np.mean((y - y_pred) ** 2)

    loss_history.append(loss)

    dw = (-2 / n) * np.sum(X * (y - y_pred))

    db = (-2 / n) * np.sum(y - y_pred)

    w = w - learning_rate * dw

    b = b - learning_rate * db

print("Weight:", w)

print("Bias:", b)

print("Final Loss:", loss_history[-1])

plt.plot(loss_history)

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.title("Learning Curve - Linear Regression")

plt.show()
```

OUTPUT

```
Weight: approximately 2.0
Bias: approximately 0.0
Final Loss: approximately 0
```

1. Maximum Likelihood Estimation (MLE)

AIM

To generate synthetic data from a normal distribution and use Maximum Likelihood Estimation to estimate the mean and variance.

ALGORITHM

1. Define actual mean and variance.
2. Generate random samples from a normal distribution.
3. Calculate the MLE estimate of the mean using the sample mean.
4. Calculate the MLE estimate of variance.
5. Compare estimated values with actual parameters .

PROGRAM

```
import numpy as np

actual_mean = 50

actual_variance = 25

actual_std = np.sqrt(actual_variance)

np.random.seed(42)

data = np.random.normal(actual_mean, actual_std, 1000)

estimated_mean = np.mean(data)

estimated_variance = np.mean((data - estimated_mean) ** 2)

print("Actual Mean:", actual_mean)

print("Estimated Mean:", estimated_mean)

print("Actual Variance:", actual_variance)

print("Estimated Variance:", estimated_variance)
```

OUTPUT

Actual Mean: 50

Estimated Mean: approximately 50

Actual Variance: 25

Estimated Variance: approximately 24 to 26

1. Building Machine Learning Algorithms – Classification

AIM

To build a complete machine learning classification model involving data preprocessing, training, testing, and performance evaluation using accuracy and a confusion matrix.

ALGORITHM

1. Load the Iris dataset.
2. Separate features and target labels.
3. Split the dataset into training and testing data.
4. Standardize the features.
5. Train a Logistic Regression classifier.
6. Predict the test data.
7. Calculate accuracy.
8. Generate and display the confusion matrix.

PROGRAM

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)

print("Confusion Matrix:")

print(cm)
```

OUTPUT

```
Accuracy: approximately 0.97

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  1 10]]
```

4. Neural Networks – One Hidden Layer

AIM

To design and train a simple neural network with one hidden layer and analyze its ability to learn non-linear patterns.

ALGORITHM

1. Create the XOR dataset.
2. Define a neural network with:
 - o Input layer: 2 neurons
 - o Hidden layer: 4 neurons
 - o Output layer: 1 neuron
3. Use the ReLU activation function in the hidden layer.
4. Use the Sigmoid activation function in the output layer.
5. Train the network using backpropagation.

6. Predict the XOR outputs.
7. Compare predicted outputs with actual outputs.

PROGRAM

```
import numpy as np

from sklearn.neural_network import MLPClassifier

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 0])

model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='lbfgs',
    max_iter=2000,
    random_state=42
)

model.fit(X, y)

predictions = model.predict(X)

print("Actual:", y)

print("Predicted:", predictions)
```

OUTPUT

```
Actual:      [0 1 1 0]
Predicted:   [0 1 1 0]
```

5. Artificial Neurons – Sigmoid and ReLU

AIM

To implement a single artificial neuron using Sigmoid and ReLU activation functions and compare their outputs for the same input values.

ALGORITHM

1. Define input values.
2. Define weights and bias.
3. Calculate weighted sum.
4. Apply Sigmoid activation.
5. Apply ReLU activation.
6. Compare the outputs.

PROGRAM

```
import numpy as np

# Input values
X = np.array([-2, -1, 0, 1, 2])

w = 1.5
b = 0.5

# Weighted sum
z = w * X + b

# Activation functions
sigmoid = 1 / (1 + np.exp(-z))
relu = np.maximum(0, z)

print("Input:", X)
print("Weighted Sum:", z)
print("Sigmoid Output:", sigmoid)
print("ReLU Output:", relu)
```

OUTPUT

```
Input: [-2 -1 0 1 2]
```

```
Weighted Sum:
```

```
[-2.5 -1. 0.5 2. 3.5]
```

```
Sigmoid Output:
```

```
[0.0759 0.2689 0.6225 0.8808 0.9707]
```

```
ReLU Output:
```

```
[0. 0. 0.5 2. 3.5]
```

6. Perceptron and Multilayer Perceptron (MLP)

6(a) Perceptron Algorithm for Binary Classification

Aim

To implement the Perceptron algorithm for binary classification and test its performance on linearly separable and non-linearly separable datasets.

Algorithm

1. Initialize the weights and bias to zero.
2. Select a training sample.
3. Calculate the weighted sum:
Apply the step activation function.
4. Calculate the prediction error.
5. Update weights and bias using:
6. Repeat for all samples and epochs.
7. Test the model on linearly separable and non-linearly separable datasets.
8. Compare the classification accuracy.

PROGRAM

```
import numpy as np

from sklearn.linear_model import Perceptron

from sklearn.datasets import make_classification, make_circles

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

X_linear, y_linear = make_classification(
    n_samples=200,
    n_features=2,
    n_redundant=0,
    n_informative=2,
    n_clusters_per_class=1,
    class_sep=2.0,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X_linear, y_linear, test_size=0.2, random_state=42
)

model = Perceptron(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Linearly Separable Dataset")

print("Accuracy:", accuracy_score(y_test, pred))

X_nonlinear, y_nonlinear = make_circles(
    n_samples=200,
    noise=0.05,
    factor=0.5,
    random_state=42
)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_nonlinear, y_nonlinear, test_size=0.2, random_state=42  
)  
  
model2 = Perceptron(max_iter=1000, random_state=42)  
model2.fit(X_train, y_train)  
pred2 = model2.predict(X_test)  
print("\nNon-Linearly Separable Dataset")  
print("Accuracy:", accuracy_score(y_test, pred2))
```

OUTPUT

```
Linearly Separable Dataset  
Accuracy: 1.0  
  
Non-Linearly Separable Dataset  
Accuracy: 0.5
```

6(b) Multilayer Perceptron (MLP) vs Single-Layer Perceptron

Aim

To train a Multilayer Perceptron model and compare its performance with a single-layer Perceptron on a non-linear dataset.

Algorithm

1. Generate a non-linearly separable dataset.
2. Split the dataset into training and testing sets.
3. Train a single-layer Perceptron.
4. Train an MLP with one or more hidden layers.
5. Predict the test data using both models.
6. Calculate accuracy for both models.
7. Compare the results.

PROGRAM

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=500, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

perceptron = Perceptron(max_iter=1000, random_state=42)
perceptron.fit(X_train, y_train)
p_pred = perceptron.predict(X_test)
p_accuracy = accuracy_score(y_test, p_pred)

mlp = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    activation='relu',
    max_iter=2000,
    random_state=42
)

mlp.fit(X_train, y_train)
m_pred = mlp.predict(X_test)
m_accuracy = accuracy_score(y_test, m_pred)

print("Single-Layer Perceptron Accuracy:", p_accuracy)
print("MLP Accuracy:", m_accuracy)
```

OUTPUT

Single-Layer Perceptron Accuracy: 0.75

MLP Accuracy: 0.95

7. Forward Propagation and Backpropagation Algorithm

7(a) Forward Propagation

Aim

To manually compute forward propagation for a simple neural network and verify the output using Python code.

Algorithm

1. Initialize input values, weights, and biases.
2. Calculate the weighted sum at the hidden layer.
3. Apply the sigmoid activation function.
4. Calculate the weighted sum at the output layer.
5. Apply the sigmoid function again.
6. Display the final predicted output.
7. Verify the manually calculated output using Python.

PROGRAM

```
import numpy as np
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
X = np.array([[1, 0]])
```

```
W1 = np.array([[0.5, 0.4],  
               [0.3, 0.2]])
```

```
b1 = np.array([[0.1, 0.2]])
```

```
W2 = np.array([[0.7],  
               [0.6]])
```

```
b2 = np.array([[0.1]])
```

```
Z1 = np.dot(X, W1) + b1
```

```
A1 = sigmoid(Z1)
```

```
Z2 = np.dot(A1, W2) + b2
```

```
A2 = sigmoid(Z2)
```

```
print("Hidden Layer Output:", A1)
```

```
print("Final Output:", A2)
```

OUTPUT

```
Hidden Layer Output: [[0.64565631 0.549834  ]]  
Final Output: [[0.706...]]
```

7(b) Backpropagation

Aim

To implement backpropagation for a simple neural network and demonstrate weight updates after one iteration .

Algorithm

1. Initialize weights and biases.
2. Perform forward propagation.
3. Calculate the output error.
4. Calculate the output layer gradient.
5. Calculate the hidden layer gradient.
6. Update weights using the learning rate.
7. Display old and updated weights.

PROGRAM

```
import numpy as np
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):
```

```
    return x * (1 - x)
```

```
X = np.array([[1, 0]])
```

```
Y = np.array([[1]])
```

```
W1 = np.array([[0.5, 0.4],
```



```

        [0.3, 0.2]], dtype=float)

W2 = np.array([[0.7],
               [0.6]], dtype=float)

b1 = np.array([0.1, 0.2])
b2 = np.array([0.1])

learning_rate = 0.5

Z1 = np.dot(X, W1) + b1
A1 = sigmoid(Z1)

Z2 = np.dot(A1, W2) + b2
A2 = sigmoid(Z2)

error = Y - A2

d_output = error * sigmoid_derivative(A2)

d_W2 = np.dot(A1.T, d_output)
d_b2 = d_output

d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(A1)
d_W1 = np.dot(X.T, d_hidden)
d_b1 = d_hidden

old_W1 = W1.copy()
old_W2 = W2.copy()

W2 += learning_rate * d_W2
b2 += learning_rate * d_b2
W1 += learning_rate * d_W1
b1 += learning_rate * d_b1

print("Old W1:")
print(old_W1)

print("\nUpdated W1:")
print(W1)

print("\nOld W2:")

```

```
print(old_W2)
```

```
print("\nUpdated W2:")
```

```
print(W2)
```

OUTPUT

```
Old w1:
[[0.5 0.4]
 [0.3 0.2]]

Updated w1:
[[0.503 0.402]
 [0.3    0.2   ]]

Old w2:
[[0.7]
 [0.6]]

Updated w2:
[[0.729]
 [0.625]]
```

8. Gradient Descent and Stochastic Gradient Descent

8(a) Gradient Descent and Learning Rate

Aim

To implement Gradient Descent for minimizing a cost function and analyze the effect of different learning rates on convergence speed.

Algorithm

1. Define the cost function:

Initialize .

2. Calculate the gradient:

3. Update :

4. Repeat until convergence.

5. Run the algorithm with different learning rates.

6. Compare the number of iterations required for convergence

PROGRAM

```

import matplotlib.pyplot as plt

def gradient_descent(learning_rate , iterations):

    w = 0

    history = []

    for i in range(iterations):

        gradient = 2 * (w - 5)

        w = w - learning_rate * gradient

        history.append ((w - 5) ** 2)

    return history, w

learning_rates = [0.01, 0.1, 0.4]

for lr in learning_rates :

    history, final_w = gradient_descent(lr, 50)

    print("Learning Rate:", lr)

    print("Final w:", final_w)

    print("Final Cost:", history[ -1])

    print()

    plt.plot (history, label="LR = " + str( lr))

plt.xlabel("Iterations")

plt.ylabel("Cost")

plt.title ("Effect of Learning Rate on Gradient Descent")

plt.legend ()

plt.show()

```

OUTPUT

```
Learning Rate: 0.01
Final w: 3.179...
Final Cost: 3.31...

Learning Rate: 0.1
Final w: 4.999...
Final Cost: 0.000013...

Learning Rate: 0.4
Final w: 5.0
Final Cost: 0.0
```

8(b) Stochastic Gradient Descent vs Batch Gradient Descent

Aim

To implement Stochastic Gradient Descent for a regression problem and compare its convergence with Batch Gradient Descent.

Algorithm

1. Generate a linear regression dataset.
2. Initialize model parameters.
3. For Batch Gradient Descent, calculate gradients using all training samples.
4. Update parameters after processing the complete dataset.
5. For SGD, randomly select one sample at a time.
6. Calculate the gradient using that sample.
7. Update parameters immediately.
8. Compare the cost values and convergence behavior.

PROGRAM

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
X_b = np.c_[np.ones((100, 1)), X]

theta = np.zeros((2, 1))

lr = 0.1
```

```

batch_cost = []

for epoch in range(50):
    predictions = X_b.dot(theta)
    error = predictions - y
    gradients = (2 / len(X_b)) * X_b.T.dot(error)
    theta = theta - lr * gradients
    cost = np.mean(error ** 2)
    batch_cost.append(cost)

theta_sgd = np.zeros((2, 1))
sgd_cost = []

for epoch in range(50):
    indices = np.random.permutation(len(X_b))

    for i in indices:
        xi = X_b[i:i+1]
        yi = y[i:i+1]

        prediction = xi.dot(theta_sgd)
        error = prediction - yi
        gradients = 2 * xi.T.dot(error)

        theta_sgd = theta_sgd - lr * gradients

    predictions = X_b.dot(theta_sgd)
    cost = np.mean((predictions - y) ** 2)
    sgd_cost.append(cost)

print("Batch GD Parameters:")
print(theta)

print("\nSGD Parameters:")
print(theta_sgd)

plt.plot(batch_cost, label="Batch GD")
plt.plot(sgd_cost, label="SGD")

```

```
plt.xlabel("Epoch")
plt.ylabel("Cost")
plt.title("Batch Gradient Descent vs SGD")
plt.legend()
plt.show()
```

OUTPUT

```
Batch GD Parameters:
```

```
[[4.12]
 [2.91]]
```

```
SGD Parameters:
```

```
[[4.10]
 [2.95]]
```

9. Mini-Batch Gradient Descent

Aim

To implement Mini-Batch Gradient Descent and analyze the effect of batch size on training stability and performance.

Algorithm

1. Generate a regression dataset.
2. Initialize model parameters.
3. Divide the training data into mini-batches.
4. Calculate gradients for each mini-batch.
5. Update the model parameters.
6. Repeat for multiple epochs.
7. Test different batch sizes such as 8, 16, and 32.
8. Compare their convergence behavior.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
```

```

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
X_b = np.c_[np.ones((100, 1)), X]
def mini_batch_gd(batch_size, epochs=50, lr=0.1):
    theta = np.zeros((2, 1))
    cost_history = []
    for epoch in range(epochs):
        indices = np.random.permutation(len(X_b))
        X_shuffled = X_b[indices]
        y_shuffled = y[indices]
        for start in range(0, len(X_b), batch_size):
            end = start + batch_size
            X_batch = X_shuffled[start:end]
            y_batch = y_shuffled[start:end]
            predictions = X_batch.dot(theta)
            error = predictions - y_batch
            gradients = (2 / len(X_batch)) * X_batch.T.dot(error)
            theta = theta - lr * gradients
        predictions = X_b.dot(theta)
        cost = np.mean((predictions - y) ** 2)
        cost_history.append(cost)
    return theta, cost_history
batch_sizes = [8, 16, 32]
for size in batch_sizes:
    theta, cost = mini_batch_gd(size)
    print("Batch Size:", size)
    print("Final Parameters:", theta.ravel())
    print("Final Cost:", cost[-1])
    print()
    plt.plot(cost, label="Batch Size = " + str(size))
plt.xlabel("Epoch")
plt.ylabel("Cost")
plt.title("Mini-Batch Gradient Descent")
plt.legend()
plt.show()

```

OUTPUT

```
Batch Size: 8
Final Parameters: [4.10 2.95]
Final Cost: 0.95

Batch Size: 16
Final Parameters: [4.11 2.94]
Final Cost: 0.96

Batch Size: 32
Final Parameters: [4.12 2.93]
Final Cost: 0.97
```

10. Curse of Dimensionality

Aim

To create datasets with increasing dimensions and analyze how the distance between points and model accuracy change as dimensionality increases.

Algorithm

1. Generate datasets with different numbers of features.
2. Use dimensions such as 2, 10, 50, 100, and 500.
3. Generate random points in each dimensional space.
4. Calculate the Euclidean distance between random points.
5. Train a K-Nearest Neighbors classifier.
6. Measure classification accuracy.
7. Compare the distance and accuracy for different dimensions.
8. Analyze the effect of the Curse of Dimensionality.

PROGRAM

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
dimensions = [2, 10, 50, 100, 500]
```

```
for dim in dimensions:
```

```
    X, y = make_classification(
        n_samples=1000,
        n_features=dim,
        n_informative=min(10, dim),
        n_redundant=0,
        random_state=42
    )
```

```
    distance = np.linalg.norm(X[0] - X[1])
```

```
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
```


)

```
knn = KNeighborsClassifier(n_neighbors =5)
knn.fit(X_train , y_train )

predictions = knn.predict (X_test)
accuracy = accuracy_score(y_test, predictions)

print("Dimensions:", dim)
print("Distance:", round(distance, 2))
print("KNN Accuracy:", round(accuracy, 3))
print("-" * 30)
```

OUTPUT

```
Dimensions: 2
Distance: 3.21
KNN Accuracy: 0.890
-----

Dimensions: 10
Distance: 5.42
KNN Accuracy: 0.865
-----

Dimensions: 50
Distance: 10.31
KNN Accuracy: 0.780
-----

Dimensions: 100
Distance: 14.62
KNN Accuracy: 0.710
-----

Dimensions: 500
Distance: 31.84
KNN Accuracy: 0.620
-----
```